

Deployment of a Serverless Function Using AWS Lambda and API Gateway

Task 3 – Cloud Computing Practical Report

Submitted by: Habiba Shafait

Alfido Tech

1. Objective

The objective of this task is to create and deploy a serverless function using AWS Lambda, test its execution within the AWS console, and expose it publicly as an HTTP endpoint by configuring an Amazon API Gateway trigger. This exercise demonstrates practical skills in serverless computing, function-as-a-service (FaaS) deployment, and building simple API-driven backends without managing any underlying servers.

2. Tools and Environment Used

- Amazon Web Services (AWS) Management Console – Lambda Service
- Runtime: Python 3.14
- Trigger: Amazon API Gateway (REST API)
- Region: US East (N. Virginia)
- Code Editor: AWS Lambda in-console code editor
- Access Method: Web Browser (AWS Console) and public API endpoint

3. Step-by-Step Procedure

Step 1: Navigating to the AWS Lambda Service

The AWS Management Console was searched for 'Lambda', which is the AWS service used to run code without provisioning or managing servers.

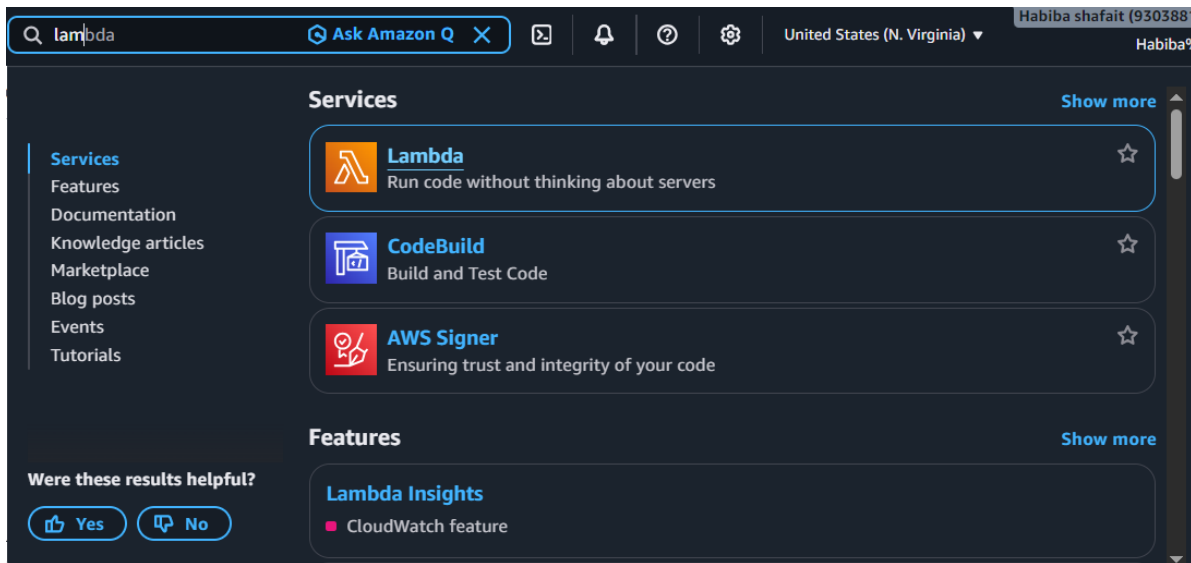


Figure 1: Searching for the AWS Lambda service in the console

Step 2: Opening the AWS Lambda Landing Page

The AWS Lambda landing page was opened, which allows functions to be created either from scratch, from a blueprint, or from a container image. The 'Create a function' option was selected to begin.

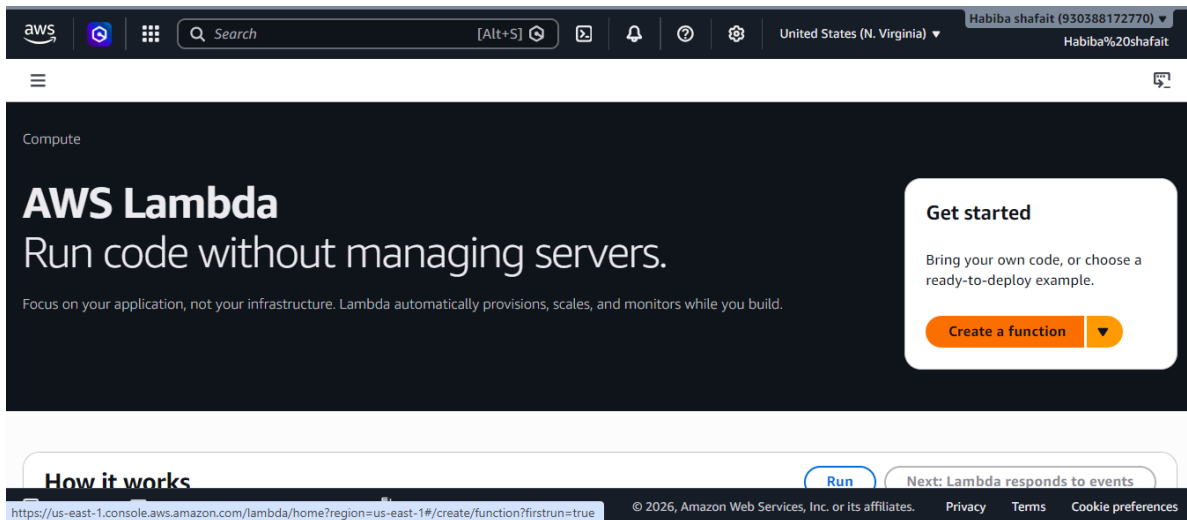


Figure 2: AWS Lambda service landing page

Step 3: Creating a New Function

The 'Author from scratch' option was chosen, and the function was named 'MYFirstFunction'. This option starts the function with a simple Hello World example, which was later customized.

Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

MYFirstFunction

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Info **Tutorials** >

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app ^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage

Figure 3: Creating a new function from scratch with the name 'MYFirstFunction'

Step 4: Selecting the Runtime

The runtime environment for the function was set to Python 3.14. AWS Lambda automatically creates an execution role with permissions to upload logs to Amazon CloudWatch by default.

≡ [Lambda](#) > [Functions](#) > **Create function**

Runtime | [Info](#)

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.14 ▼

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

Figure 4: Selecting Python 3.14 as the function runtime

Step 5: Reviewing Additional Function Settings

Before finalizing creation, the additional configuration options were reviewed, including durable execution and EC2 capacity provider settings, which allow customization of the function's execution behavior.

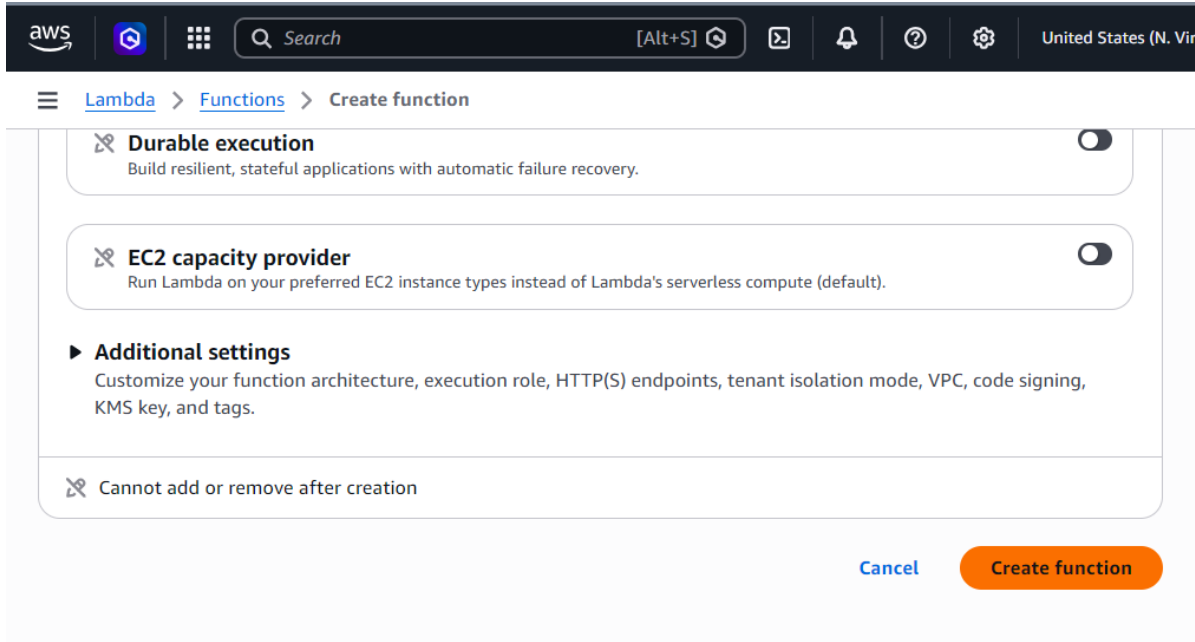


Figure 5: Reviewing additional Lambda function settings

Step 6: Verifying Successful Function Creation

After creation, the function overview page for 'MYFirstFunction' was displayed, confirming the function's ARN, description, and that it was ready for configuration and coding.

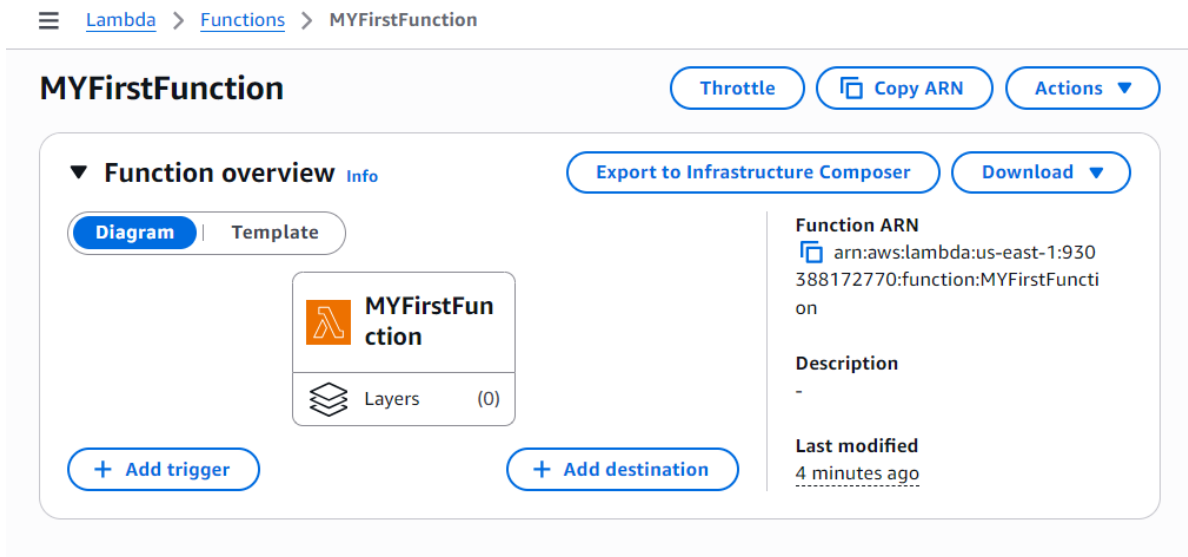


Figure 6: MYFirstFunction overview page confirming successful creation

Step 7: Writing the Function Code

The default 'lambda_function.py' file was edited in the built-in code source editor. The function was written to return a JSON response containing a greeting message and a success status, using Python's 'json' module.

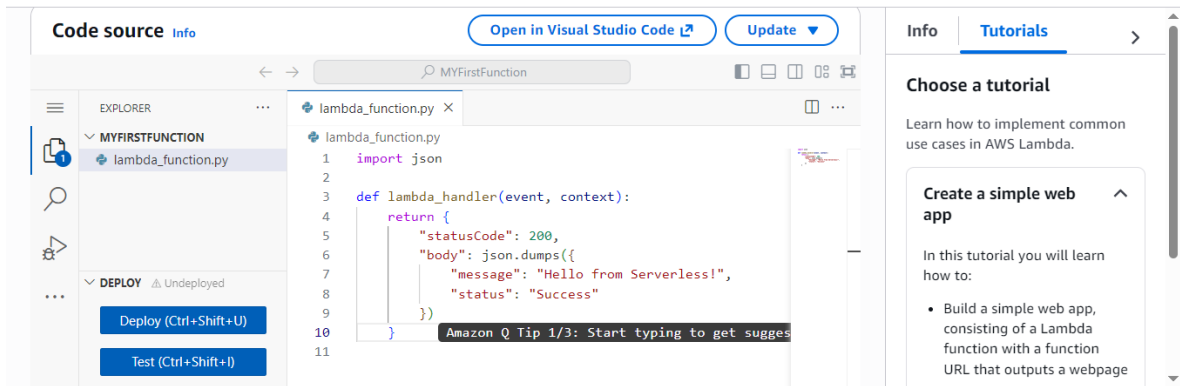


Figure 7: Lambda function source code in the in-console editor

Step 8: Deploying the Function Code

The code changes were saved and deployed using the 'Deploy' button. A confirmation message, 'Successfully updated the function "MYFirstFunction"', verified that the new code was active.

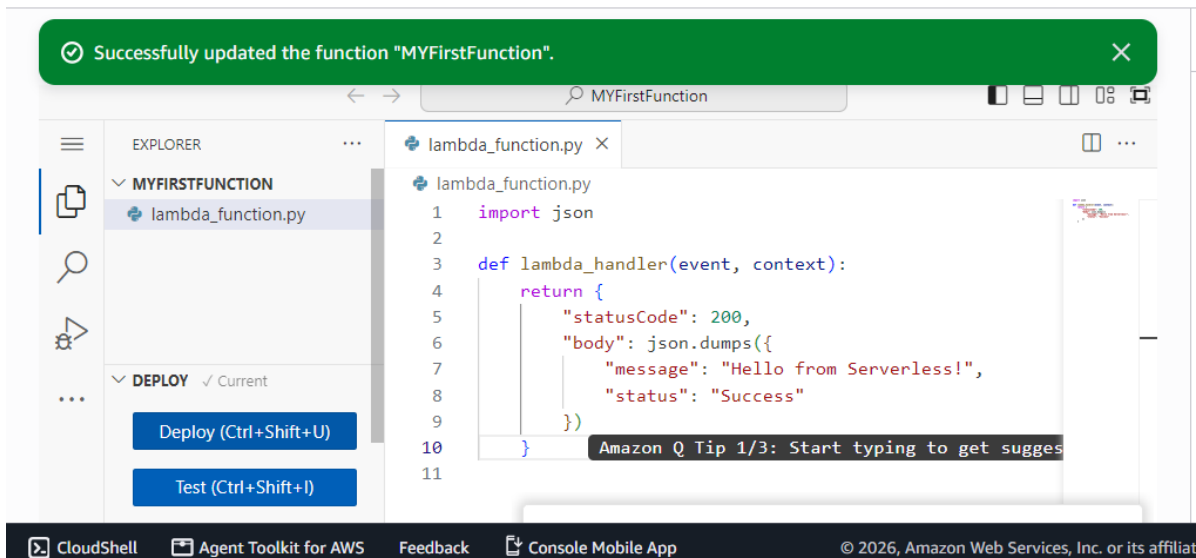


Figure 8: Confirmation that the function code was successfully deployed

Step 9: Creating a Test Event

A new test event named 'TestEvent' was created using the console's built-in testing tool, configured with a synchronous invocation type to directly return the function's output.

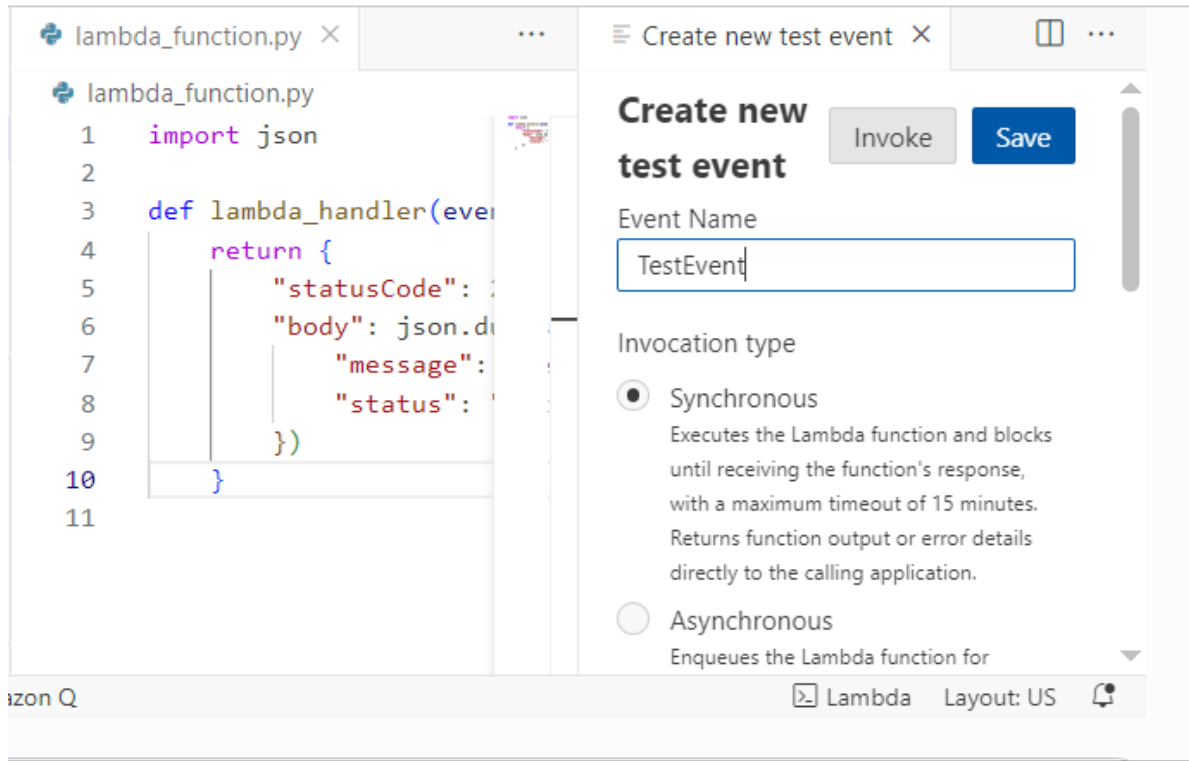


Figure 9: Creating a new test event to invoke the function

Step 10: Executing the Test and Verifying the Output

The test was executed, and the execution results showed 'Status: Succeeded', confirming that the function ran correctly and returned the expected JSON response.

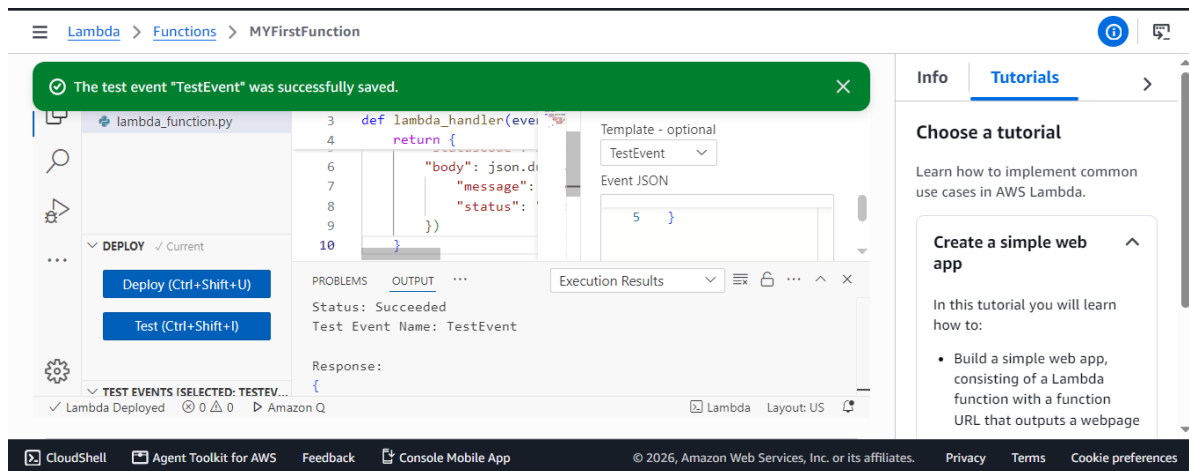


Figure 10: Successful test execution with 'Status: Succeeded'

Step 11: Adding an API Gateway Trigger

An API Gateway trigger was added to the function to expose it as a public HTTP endpoint. A new REST API was created for this purpose, allowing external clients to invoke the Lambda function over HTTP.

Add trigger

Trigger configuration [Info](#)

API Gateway
aws api application-services backend HTTP REST serverless

Add an API to your Lambda function to create an HTTP endpoint that invokes your function. API Gateway supports REST, HTTP, and WebSocket APIs. [Learn more](#)

Intent
Use an existing api or have us create one for you.

☒ Create a new API
☐ Use existing API

API type

Figure 11: Adding an API Gateway trigger to the Lambda function

Step 12: Configuring API Security Settings

The security configuration for the API endpoint was set to 'Open', meaning the endpoint could be invoked without additional authentication, suitable for this demonstration.

Lambda > Add triggers

REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

persistent connections for real-time use cases such as chat applications or dashboards.

complete control over the request and response along with API management capabilities.

Security
Configure the security mechanism for your API endpoint.

Open

► **Additional settings**

Lambda will add the necessary permissions for Amazon API Gateway to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

[Cancel](#) [Add](#)

Figure 12: Configuring the API Gateway security settings

Step 13: Verifying the Trigger and API Endpoint

Once the trigger was created, the function's Triggers panel displayed the newly added API Gateway trigger along with its public API endpoint URL, confirming the integration was successful.

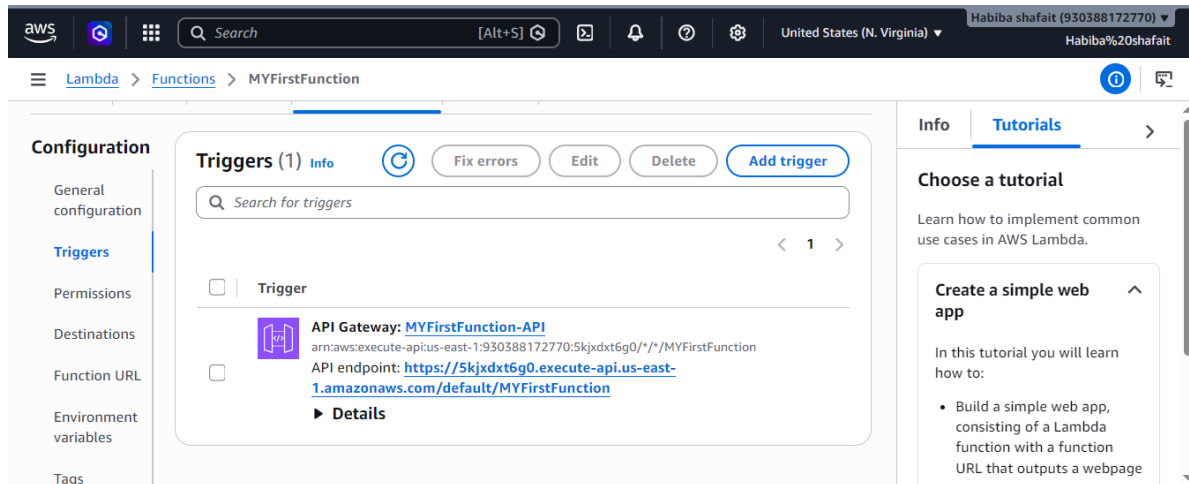


Figure 13: API Gateway trigger successfully added, showing the public endpoint URL

Step 14: Accessing the Deployed Function via the API Endpoint

Finally, the API endpoint URL was opened in a web browser. The response `{ "message": "Hello from Serverless!", "status": "Success" }` confirmed that the Lambda function was successfully deployed and publicly accessible via API Gateway.

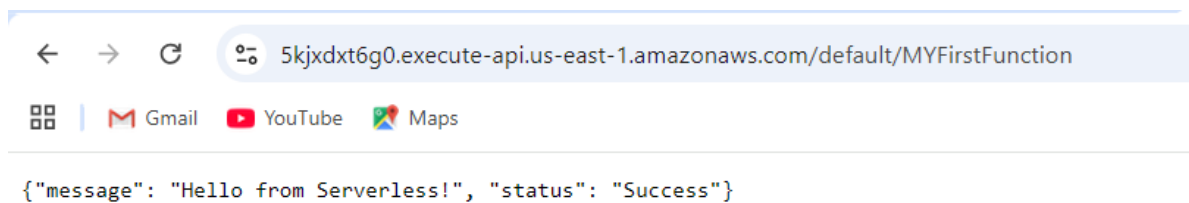


Figure 14: Browser output confirming the Lambda function response via the public API endpoint

4. Conclusion

In this task, a serverless function was successfully created and deployed using AWS Lambda with a Python 3.14 runtime. The function's code was written, deployed, and verified through a test event executed directly in the AWS console. An API Gateway trigger was then configured to expose the function as a public HTTP endpoint, which was successfully accessed via a web browser and returned the expected JSON response. This exercise provided hands-on experience with serverless computing, function-as-a-service deployment, and building simple API-driven backends — core skills for modern cloud computing and backend development roles.